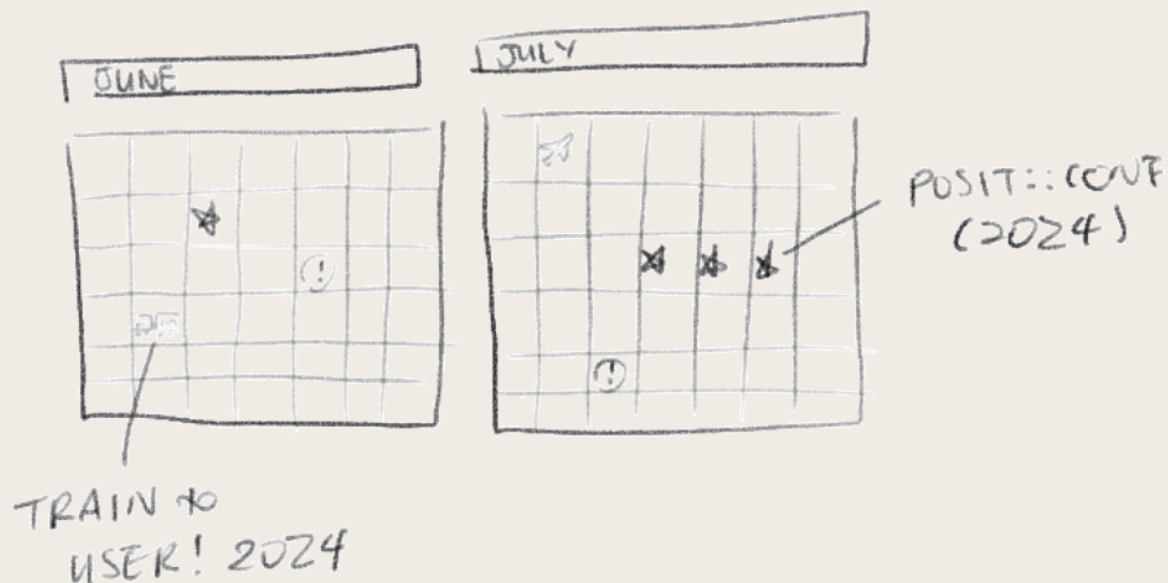


Reuse your custom ggplot2 with transparent helper functions!

Cynthia A. Huang



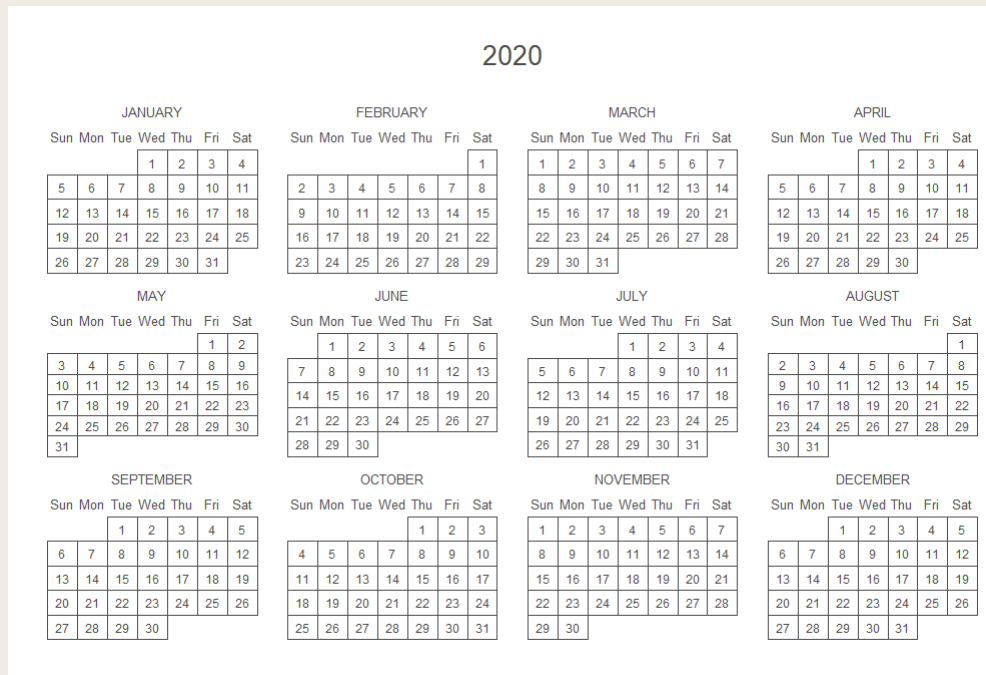
CYNTHIA'S TRAVEL CALENDAR





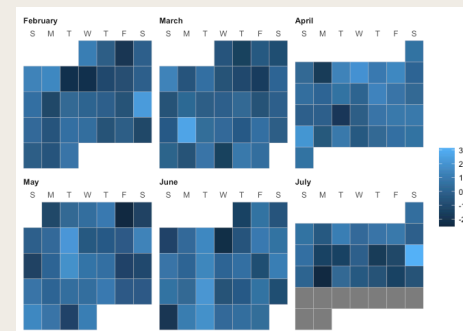
🔍 ggplot2 calendar

{calendR}



{ggweekly}

{ggcal}



{calendR}

calendR-interface

```
1  calendR(  
2    year = format(Sys.Date(), "%Y"),  
3    month = NULL,  
4    from = NULL,  
5    to = NULL,  
6    start = c("S", "M"),  
7    orientation = c("portrait", "land"),  
8    title,  
9    title.size = 20,  
10   title.col = "gray30",  
11   subtitle = "",  
12   subtitle.size = 10,  
13   subtitle.col = "gray30",  
14   text = NULL,  
15   text.pos = NULL,  
16   text.size = 4,  
17   text.col = "gray30",  
18   special.days = NULL,  
19   special.col = "gray90",  
20   gradient = FALSE,  
21   low.col = "white",  
22   col = "gray30",  
23   lwd = 0.5,  
24   lty = 1,  
25   font.family = "sans",  
26   font.style = "plain",  
27   dav.size = 3.
```

{ggweekly}

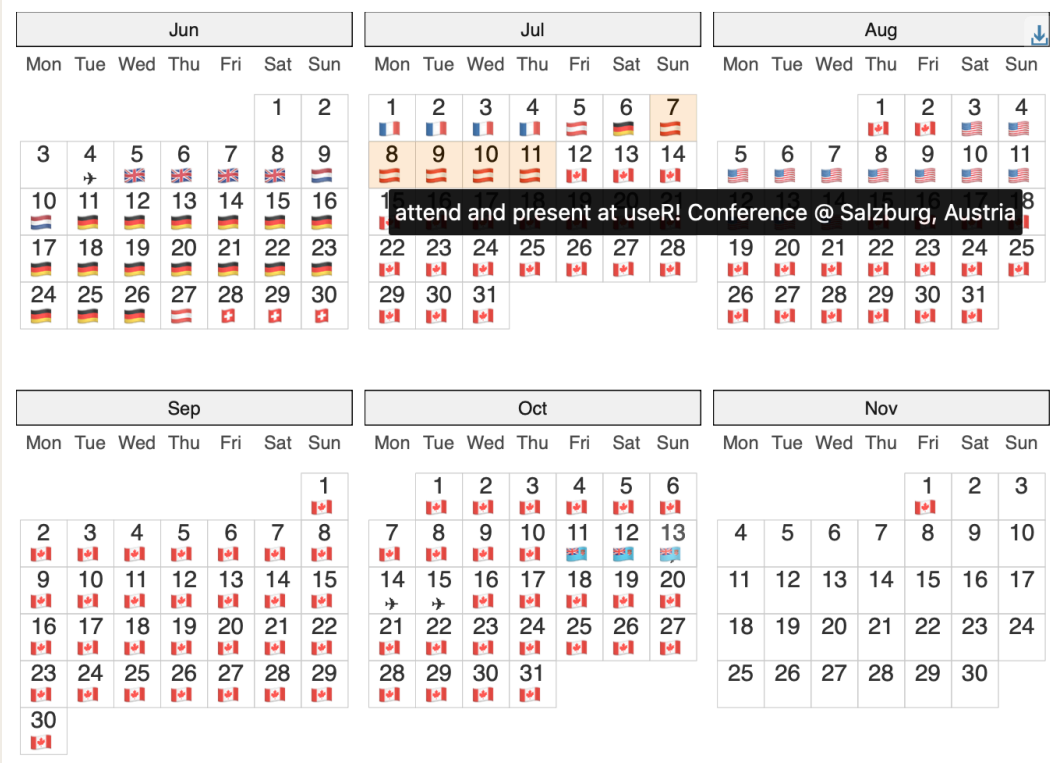
ggweekly-interface

```
1  ggweek_planner(  
2    start_day = lubridate::today(),  
3    end_day = start_day +  
4      lubridate::weeks(8) - lubridate::today(),  
5    highlight_days = NULL,  
6    week_start = c("isoweek", "epiweek"),  
7    week_start_label = c("month day", "week number"),  
8    show_day_numbers = TRUE,  
9    show_month_start_day = TRUE,  
10   show_month_boundaries = TRUE,  
11   highlight_text_size = 2,  
12   month_text_size = 4,  
13   day_number_text_size = 2,  
14   month_color = "#f78154",  
15   day_number_color = "grey80",  
16   weekend_fill = "#f8f8f8",  
17   holidays = ggweekly::us_federal_holidays,  
18   font_base_family = "PT Sans",  
19   font_label_family = "PT Sans Narr",  
20   font_label_text = NULL,  
21 )
```

{ggcal}

ggcal-interface

```
1  ggcal(dates, fills)
```

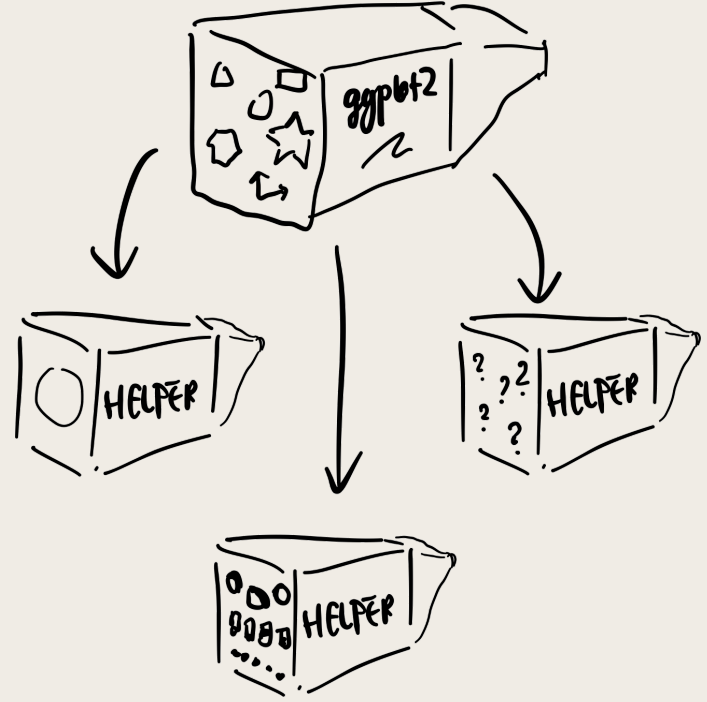


```

very-long-ggplot-script.R

1 # source(here::here("_utilities/travel_dates_load.R"))
2 library(ggplot2)
3 library(ggiraph)
4
5 ## ---- daily-loc-prep ----
6
7 ## expand dates: https://stackoverflow.com/a/54728153
8
9 travel_days <- travel_dates |>
10   mutate(nights = interval(startDate, endDate) / days(1))
11   arrange(startDate, desc(nights)) |>
12   rowid_to_column() |>
13   rowwise() |>
14   reframe(rowid, location, details, nights,
15           date = seq(startDate, endDate, by = "day"))
16   ) |>
17   group_by(date) |>
18   slice_min(order_by = nights, n = 1) |>
19   ungroup() |>
20   mutate(country = countrycode::countryname(location, "iso2c"))
21
22 away_days <- travel_days |>
23   summarise(
24     startDate = min(date),
25     endDate = max(date)
26   ) |>
27   mutate(rowid = 0, location = "TBC") |>
28   reframe(rowid, location, date = seq(startDate, endDate, by = "day"))
29
30 daily_loc <- away_days |>
31   anti_join(travel_days, by = "date") |>
32   bind_rows(travel_days) |>
33   mutate(
34     flag = countrycode::countrycode(country, "iso2c")
35   )

```



Down the function design
rabbit hole...

JUL							AUG						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
				1	2								
3	4	5	6	7	8	9	10	11	12	13	14	15	16
10	11	12	13	14	15	16	17	18	19	20	21	22	23
17	18	19	20	21	22	23	24	25	26	27	28	29	30
24	25	26	27	28	29	30							
31													

SEP							OCT							NOV						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30						
23	24	25	26	27	28	29	30	31												
30	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30			
20	21	22	23	24	25	26	27	28	29	30	31									
27	28	29	30	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54

how can I keep the ELEGANCE of COMPOSITION of ggplot2 in MY helper func?



Jan					Jul					Aug				
Mon	Tue	Wed	Thu	Fri	Sat	Mon	Tue	Wed	Thu	Fri	Sat	Mon	Tue	Wed
1	2	3	4	5	6	1	2	3	4	5	6	1	2	3
7	8	9	10	11	12	7	8	9	10	11	12	7	8	9
13	14	15	16	17	18	13	14	15	16	17	18	13	14	15
19	20	21	22	23	24	19	20	21	22	23	24	19	20	21
25	26	27	28	29	30	25	26	27	28	29	30	25	26	27
31						31						31		

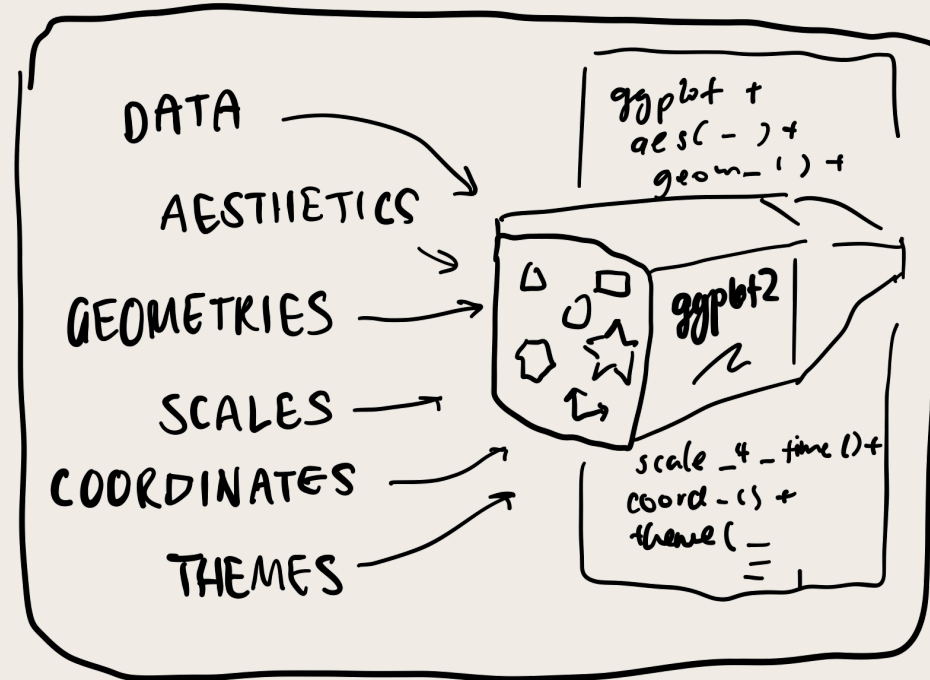
Nov					Dec					Jan				
Mon	Tue	Wed	Thu	Fri	Sat	Mon	Tue	Wed	Thu	Fri	Sat	Mon	Tue	Wed
1	2	3	4	5	6	1	2	3	4	5	6	1	2	3
7	8	9	10	11	12	7	8	9	10	11	12	7	8	9
13	14	15	16	17	18	13	14	15	16	17	18	13	14	15
19	20	21	22	23	24	19	20	21	22	23	24	19	20	21
25	26	27	28	29	30	25	26	27	28	29	30	25	26	27
31						31						31		

how can I keep
the ELEGANCE
& COMPOSITION of
ggplot2 in my helper func?

oh! JUST solve the
GRAMMAR of TEMPORAL
GRAPHICS!



HELPER () :



From procrasti-plotting
to procrasti-packaging

ggtilecal

The goal of `ggtilecal` is to easily produce calendar layouts using ggplot2 tile and text geoms, while retaining some customisation.

Installation

You can install the development version of ggtilecal like so:

```
# install.packages("remotes")
remotes::install_github("cynthiahqy/ggtilecal")
```

Examples

Empty Calendar

```
library(ggtilecal)
make_empty_month_days(c("2024-01-05", "2024-04-04")) |>
  gg_facet_wrap_months(unit_date)
```

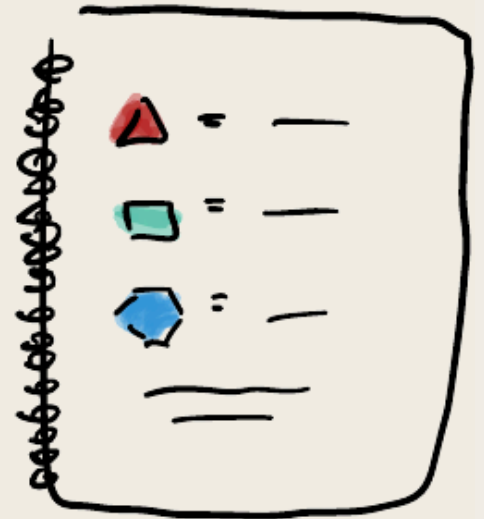
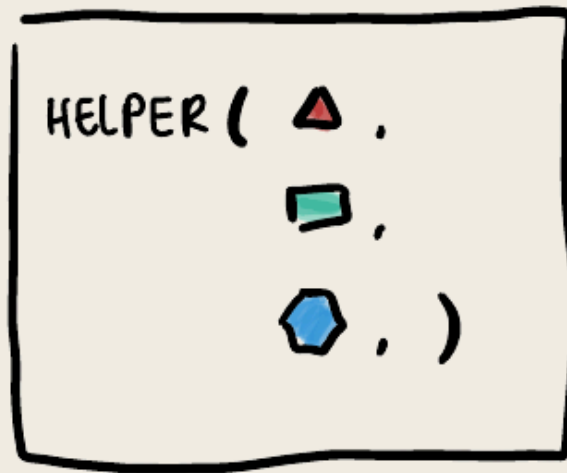
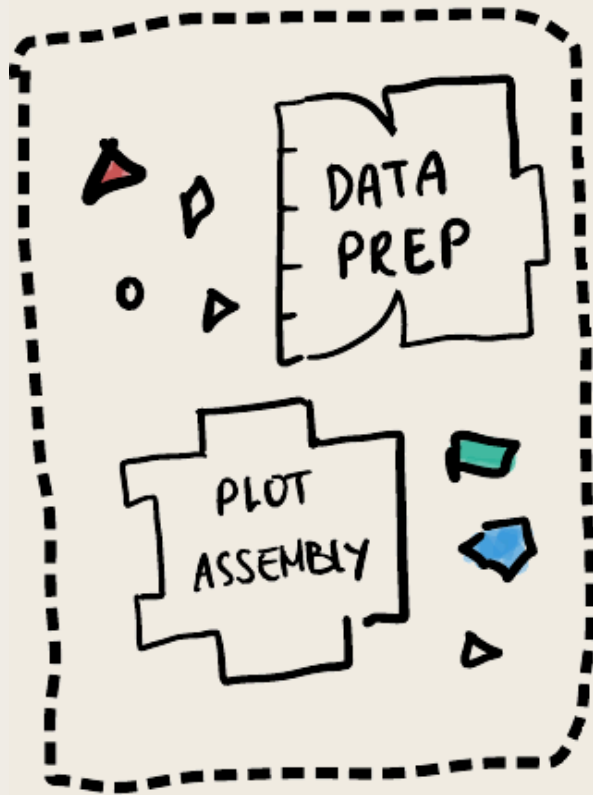
Jan							Feb						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
1	2	3	4	5	6	7				1	2	3	4
8	9	10	11	12	13	14	5	6	7	8	9	10	11
15	16	17	18	19	20	21	12	13	14	15	16	17	18
22	23	24	25	26	27	28	19	20	21	22	23	24	25
29	30	31					26	27	28	29			

A **reuseable-enough** helper should:

- **Convenient** and quick ‘autoplotting’
- Keep ggplot **flexibility** & error messages
- Play nice with **compatible** ggplot2 extensions
- **Minimise plot-specific arguments**

Design principles

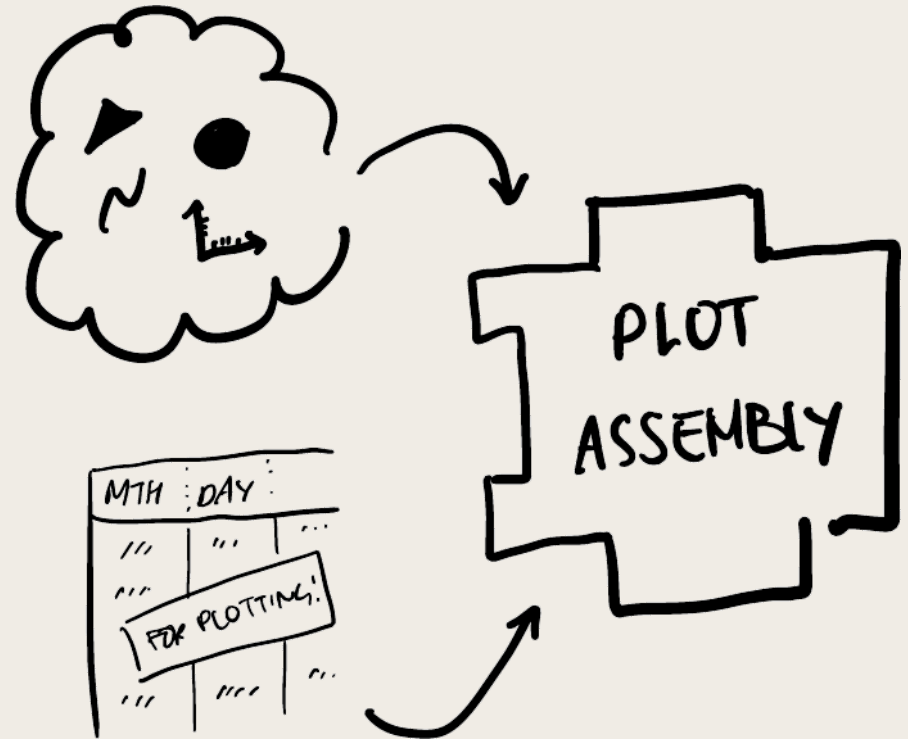
SEPARATE, EXPOSE & DOCUMENT



SEPARATE

very-long-ggplot-script.R

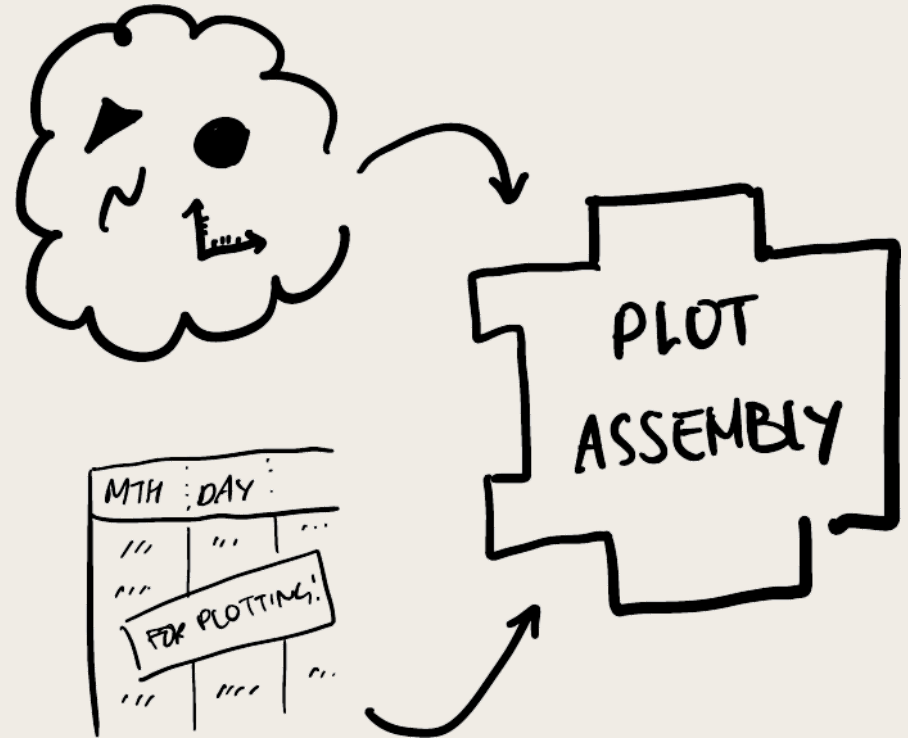
```
41   ))
42
43   ## ---- prep-full-calendar-data ----
44
45   # based on: https://github.com/nrennie/tidyuesday/blob/9dbe69d696f6c1ed
46
47   # calculate extra months to make plot rectangular
48   cal_ncol <- 3
49   startMonth <- month(floor_date(min(travel_days$date), "month"))
50   endMonth <- month(ceiling_date(max(travel_days$date), "month") - 1)
51   n_extra_month <- cal_ncol - ((endMonth - startMonth + 1) %% cal_ncol)
52   padding_days <- summarise(
53     travel_days,
54     startDate = floor_date(min(date), "month"),
55     endDate = ceiling_date(max(date) %m+% months(n_extra_month), "month")
56   ) |>
57     reframe(date = seq(startDate, endDate, by = "day"))
58
59   # prepare calendar data
60   full_calendar <- padding_days |>
61     anti_join(daily_loc, by = "date") |>
62     bind_rows(daily_loc) |>
63     mutate(
64       Year = year(date),
65       Month = month(date, label = TRUE),
66       Day = wday(date, label = TRUE, week_start = 1),
67       mday = mday(date),
68       Month_week = (5 + day(date) +
69         wday(floor_date(date, "month"), week_start = 1)) %% 7
70     )
71
72   ## ---- travel-dates-calendar-display ----
73
74   base_data_aes <- full_calendar |>
```



SEPARATE

data-prep.R

```
1 calc_calendar_vars <- function(.data, date_col,
2                               locale = Sys.getlocale("LC_TIME"),
3                               week_start = 1) {
4   .data |> dplyr::mutate(
5     TC_year = year({{ date_col }}),
6     TC_month_label = tsibble::yearmonth({{ date_col }}),
7     TC_mday = mday({{ date_col }}),
8     TC_wday_label = wday({{ date_col }},
9       label = TRUE,
10      locale = locale,
11      week_start = week_start
12    ),
13     TC_wday = wday({{ date_col }}, week_start = week_start),
14     TC_month_week = (5 + day({{ date_col }}) +
15       wday(floor_date({{ date_col }}, "month"), week_start = week_start))
16     TC_is_weekend =
17       ifelse(wday({{ date_col }}, week_start = 1) %in% c(6, 7), TRUE, FALSE)
18   )
19 }
```



SEPARATE into modular internals

HELPER():

DATA
PREP

calc_mth()
fill_dates()



PLOT
ASSEMBLY

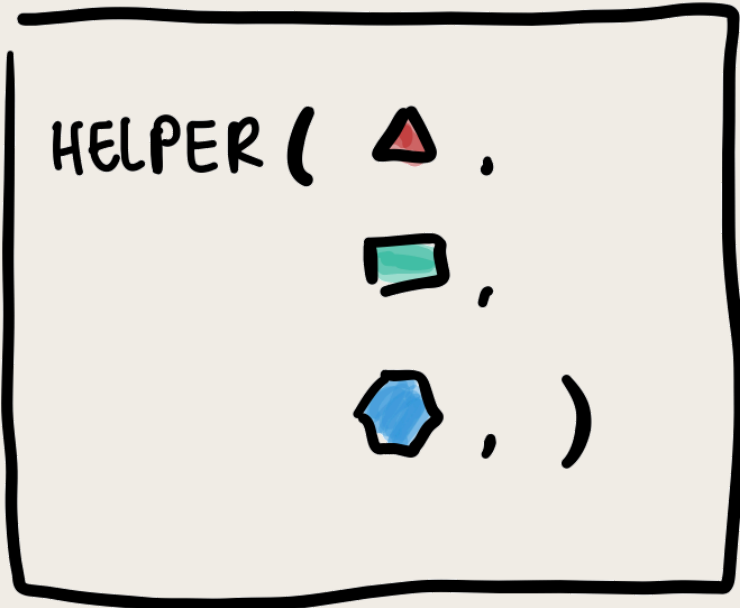
ggplot (F)
aes(~ , ~ , ~)

+ .geom
+ .scale_coord
+ .facets
+ .others

internals

```
1 gg_facet_wrap_months <- function(){
2   ## Data Prep
3   cal_data <- .events_long |>
4     fill_missing_units({{ date_col }}) |>
5     calc_calendar_vars({{ date_col }})
6
7   ## Plot Elements
8   .geom = list(
9     geom_tile(color = "grey70", fill = "transparent"),
10    geom_text(nudge_y = 0.25)
11  ),
12  .scale_coord = list(
13    scale_y_reverse(),
14    scale_x_discrete(position = "top"),
15    coord_fixed(expand = TRUE)),
16  .theme = list(theme_bw_tilecal()),
17  .other = list()
18
19  ## Plot Assembly
20  base_plot <- cal_data |>
21    ggplot2::ggplot(mapping = aes_string(
22      x = "TC_wday_label",
23      y = "TC_month_week",
24      label = "TC_mday"
25    )) +
26    facet_wrap(c("TC_month_label"), axes = "all_x", nrow = nrow, ncol = 1) +
27    labs(y = NULL, x = NULL) +
28    .geom +
29    .scale_coord +
30    .theme +
31    .other
32
33  base_plot
34 }
```

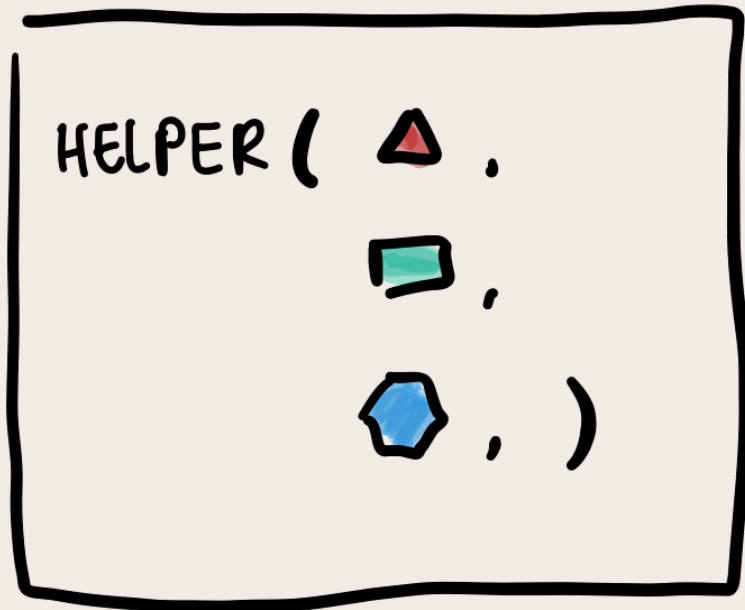
EXPOSE list defaults



function-interface

```
1 gg_facet_wrap_months(  
2   ## --- data ----  
3   .events_long,  
4   date_col,  
5   locale = NULL,  
6   ## --- facets / layout ----  
7   week_start = NULL,  
8   nrow = NULL,  
9   ncol = NULL,  
10  ## --- default layers ---  
11  .geom = list(  
12    geom_tile(color = "grey70", fill = "transpa  
13    geom_text(nudge_y = 0.25)  
14  ),  
15  .scale_coord = list(  
16    scale_y_reverse(),  
17    scale_x_discrete(position = "top"),  
18    coord_fixed(expand = TRUE)),  
19  .theme = list(theme_bw_tilecal()),  
20  .other = list()  
21 )
```

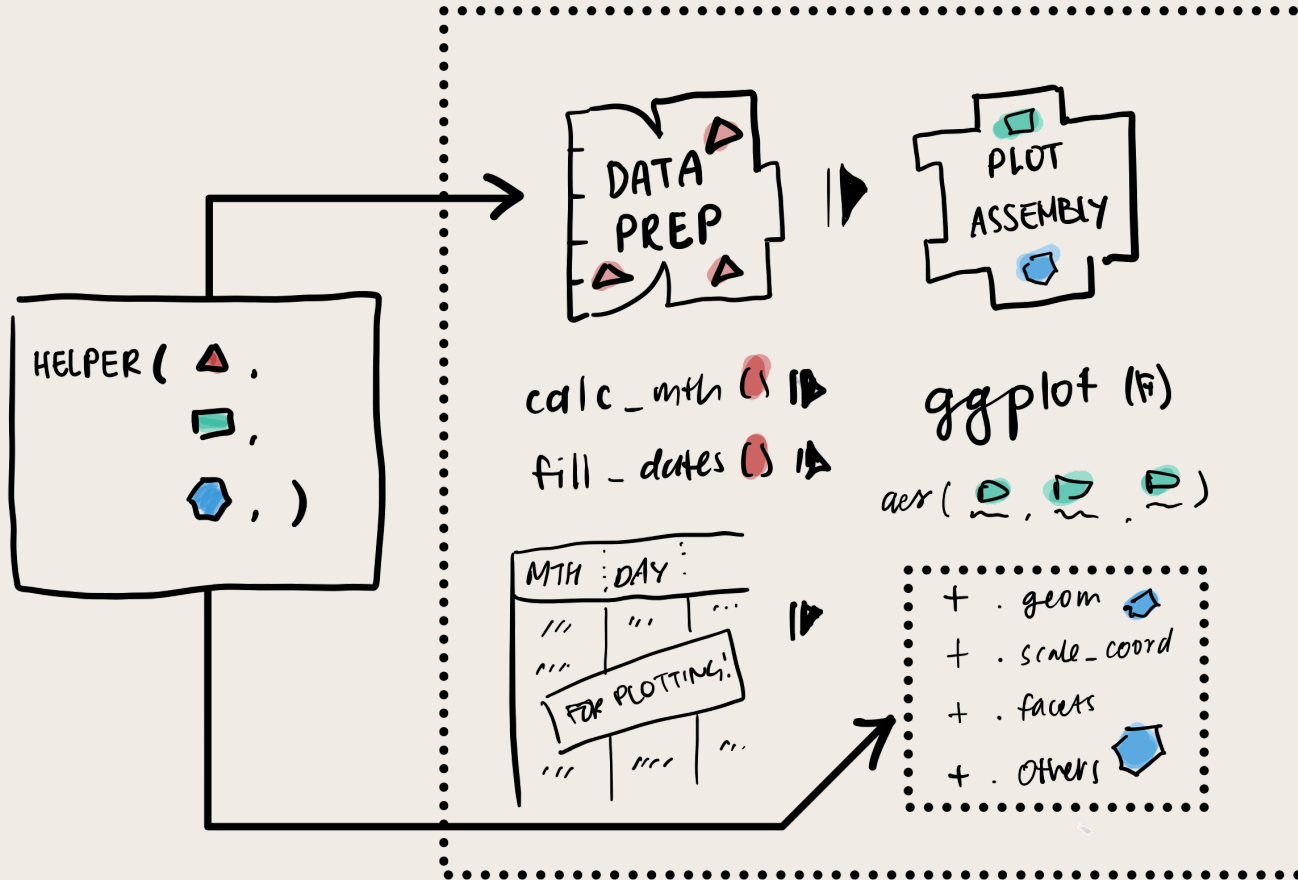
...or as chart-type arguments



function-interface

```
1 gg_facet_wrap_months(  
2   ## --- data ----  
3   .events_long,  
4   date_col,  
5   locale = NULL,  
6   ## --- facets / layout ----  
7   week_start = NULL,  
8   nrow = NULL,  
9   ncol = NULL,  
10  ## --- chart elements ---  
11  .day_tiles = list(  
12    geom_tile(color = "grey70", fill = "transpa  
13  ),  
14  .day_labels = list(  
15    geom_text(nudge_y = 0.25)  
16  ),  
17  .calendar_layout = list(  
18    scale_y_reverse(),  
19    scale_x_discrete(position = "top"),  
20    coord_fixed(expand = TRUE)),  
21  .theme = list(theme_bw_tilecal())  
22 )
```

EXPOSE



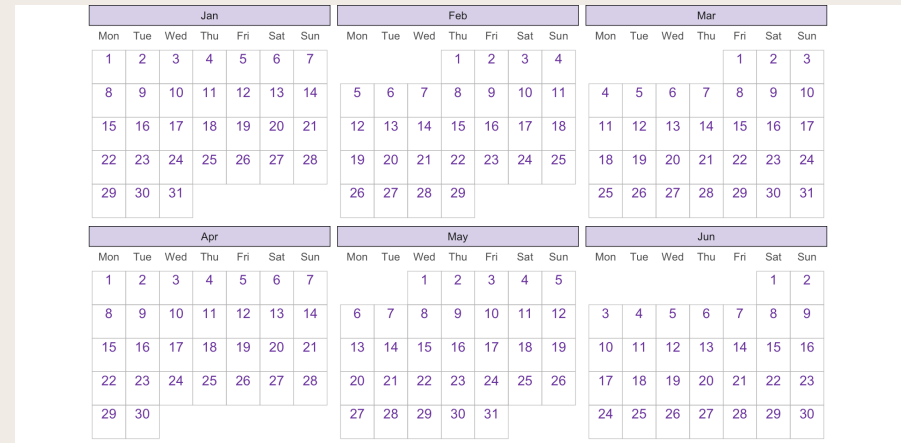
From autoplot()

```
1 library(ggtilecal)
2 library(ggplot2)
3
4 dates_df <- make_empty_month_days(
5   c("2024-01-05", "2024-06-30"), dates_to = "da
6
7 gg_facet_wrap_months(
8   dates_df,
9   date)
```

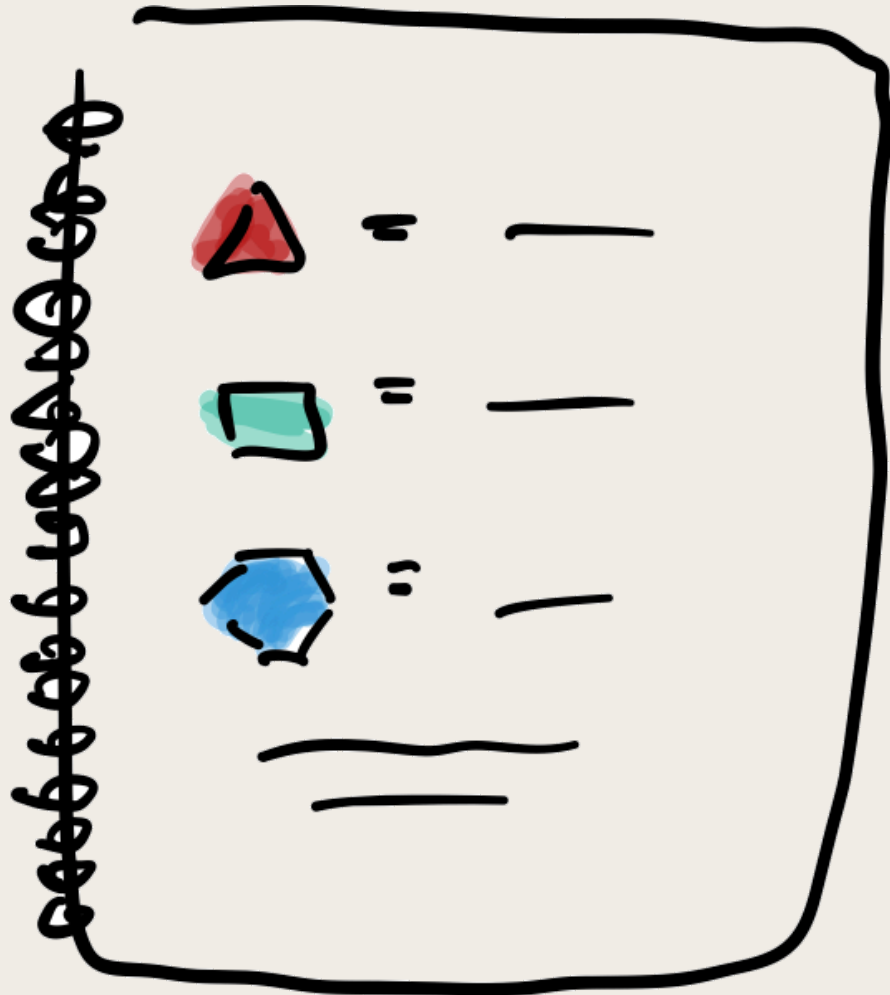


From autoplot() to customisation!

```
1 library(ggtilecal)
2 library(ggplot2)
3
4 dates_df <- make_empty_month_days(
5   c("2024-01-05", "2024-06-30"), dates_to = "day"
6 )
7 gg_facet_wrap_months(dates_df, date,
8   .geom = list(
9     geom_tile(color = "grey70", fill = "transparent"),
10    geom_text(nudge_y = 0.25,
11              color = "#6a329f")),
12   .theme = list(theme_bw_tilecal(),
13                 theme(
14                   strip.background = element_rect(fill =
15 )
```



DOCUMENT



- Data preparation steps
- Fixed variables and aesthetics
- Customisation via arguments

gg_facet_wrap_months.R

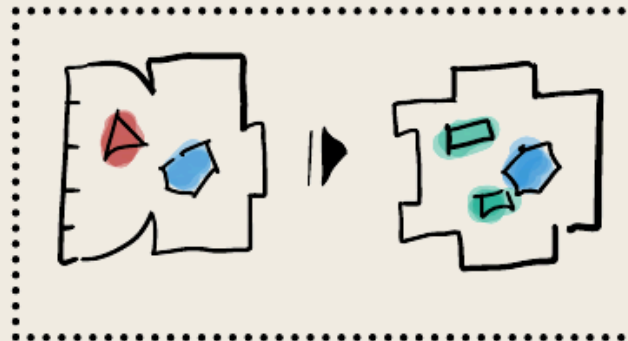
```
1 #' Make Monthly Calendar Facets
2 #'
3 #' Generates calendar with monthly facets by:
4 #' - Padding event list with any missing days via `fill_missing_units()`
5 #' - Calculating variables for calendar layout via `calc_calendar_vars()`
6 #' - Returning a ggplot object as per Details.
7 #'
8 #' Returns a ggplot with the following fixed components
9 #' using calculated layout variables:
10 #' - `aes()` mapping:
11 #'   - `x` is day of week,
12 #'   - `y` is week in month,
13 #'   - `label` is day of month
14 #' - `facet_wrap()` by month
15 #' - `labs()` to remove axis labels for calculated layout variables
16 #'
17 #' and default customisable components:
18 #' - `geom_tile()`, `geom_text()` to label each day which inherit calculate
```

Reusing happily ever
after?

From custom code...

```
very-long-ggplot-script.R
5 ## ---- daily-loc-prep ----
6
7 ## expand dates: https://stackoverflow.com/a/54728153
8
9 travel_days <- travel_dates |>
10   mutate(nights = interval(startDate, endDate) / days[1]) |>
11   arrange(startDate, desc(nights)) |>
12   rowid_to_column() |>
13   rowwise() |>
14   reframe(rowid, location, details, nights,
15     date = seq(startDate, endDate, by = "day")
16   ) |>
17   group_by(date) |>
18   slice_min(order_by = nights, n = 1) |>
19   ungroup() |>
20   mutate(country = countrycode::countryname(location, "iso2c"))
21
22 away_days <- travel_days |>
23   summarise(
24     startDate = min(date),
25     endDate = max(date)
26   ) |>
27   mutate(rowid = 0, location = "TBC") |>
28   reframe(rowid, location, date = seq(startDate, endDate, by = "day"))
29
30 daily_loc <- away_days |>
31   anti_join(travel_days, by = "date") |>
32   bind_rows(travel_days) |>
33   mutate(
34     flag = countrycode::countrycode(country, "iso2c", "unicode.symbol"),
35     continent = countrycode::countrycode(country, "iso2c", "continent")
36   ) |>
37   mutate(flag = case_when(
38     location == "en route" ~ "\u2788",
39     location == "TBC" ~ " ",
40     TRUE ~ flag
41   ))
```

HELPER():

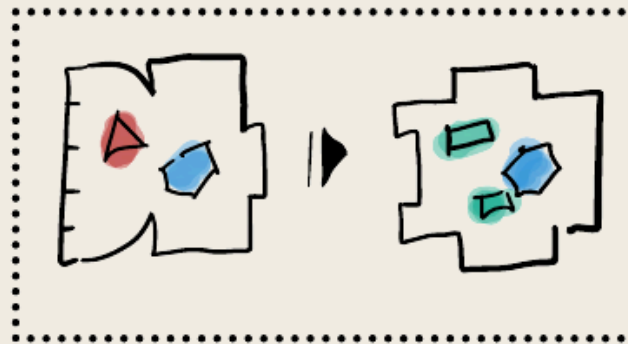


to transparent helpers...

```

very-long-ggplot-script.R
5 ## ---- daily-loc-prep ----
6
7 ## expand dates: https://stackoverflow.com/a/54728153
8
9 travel_days <- travel_dates |>
10   mutate(nights = interval(startDate, endDate) / days[1]) |>
11   arrange(startDate, desc(nights)) |>
12   rowid_to_column() |>
13   rowwise() |>
14   reframe(rowid, location, details, nights,
15           date = seq(startDate, endDate, by = "day"))
16   |>
17   group_by(date) |>
18   slice_min(order_by = nights, n = 1) |>
19   ungroup() |>
20   mutate(country = countrycode::countryname(location, "iso2c"))
21
22 away_days <- travel_days |>
23   summarise(
24     startDate = min(date),
25     endDate = max(date)
26   ) |>
27   mutate(rowid = 0, location = "TBC") |>
28   reframe(rowid, location, date = seq(startDate, endDate, by = "day"))
29
30 daily_loc <- away_days |>
31   anti_join(travel_days, by = "date") |>
32   bind_rows(travel_days) |>
33   mutate(
34     flag = countrycode::countrycode(country, "iso2c", "unicode.symbol"),
35     continent = countrycode::countrycode(country, "iso2c", "continent")
36   ) |>
37   mutate(flag = case_when(
38     location == "en route" ~ "\u2788",
39     location == "TBC" ~ " ",
40     TRUE ~ flag
  
```

HELPER() :



Jan							Feb						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
1	2	3	4	5	6	7				1	2	3	4
8	9	10	11	12	13	14	5	6	7	8	9	10	11
15	16	17	18	19	20	21	12	13	14	15	16	17	18
22	23	24	25	26	27	28	19	20	21	22	23	24	25
29	30	31					26	27	28	29			

Mar							Apr						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
				1	2	3	1	2	3	4	5	6	7
4	5	6	7	8	9	10	8	9	10	11	12	13	14
11	12	13	14	15	16	17	15	16	17	18	19	20	21
18	19	20	21	22	23	24	22	23	24	25	26	27	28
25	26	27	28	29	30	31	29	30					

Jan							Feb							Mar						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
1	2	3	4	5	6	7				1	2	3	4					1	2	3
8	9	10	11	12	13	14	5	6	7	8	9	10	11	4	5	6	7	8	9	10
15	16	17	18	19	20	21	12	13	14	15	16	17	18	11	12	13	14	15	16	17
22	23	24	25	26	27	28	19	20	21	22	23	24	25	18	19	20	21	22	23	24
29	30	31					26	27	28	29				25	26	27	28	29	30	31

Summer 2023-24

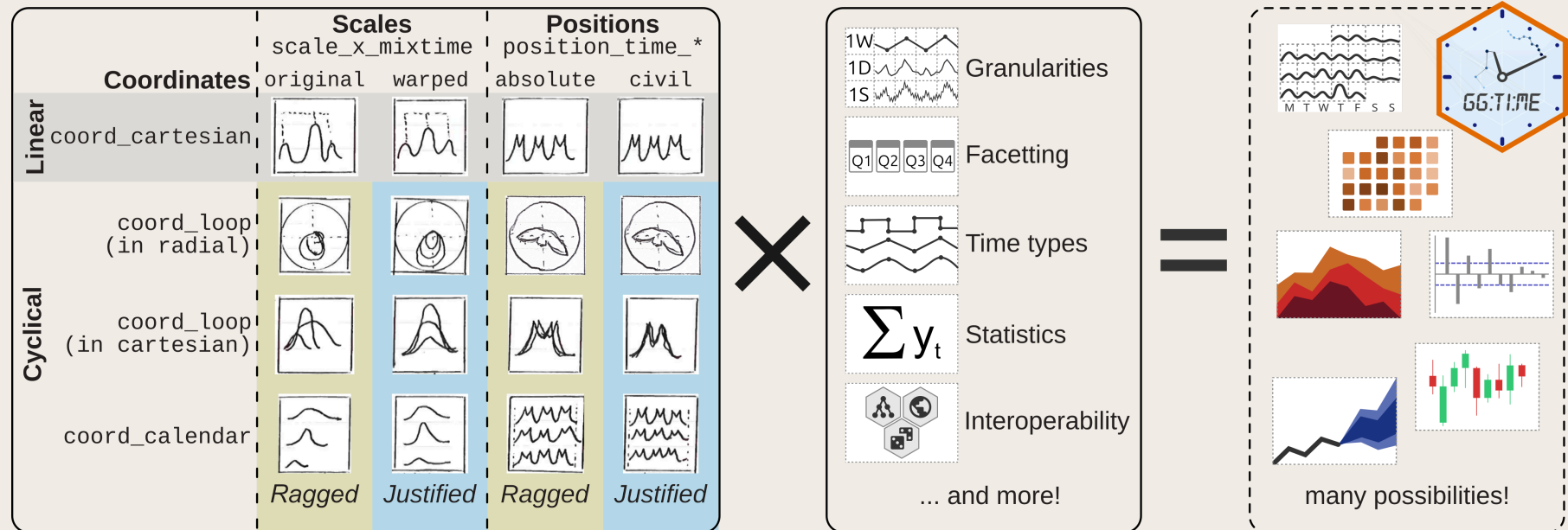
December 2023							January 2024							February 2024						
Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
				1	2	3	1	2	3	4	5	6	7				1	2	3	4
4	5	6	7	8	9	10	8	9	10	11	12	13	14	5	6	7	8	9	10	11
11	12	13	14	15	16	17	15	16	17	18	19	20	21	12	13	14	15	16	17	18
18	19	20	21	22	23	24	22	23	24	25	26	27	28	19	20	21	22	23	24	25
25	26	27	28	29	30	31	29	30	31					26	27	28	29			

and beyond?

Danger

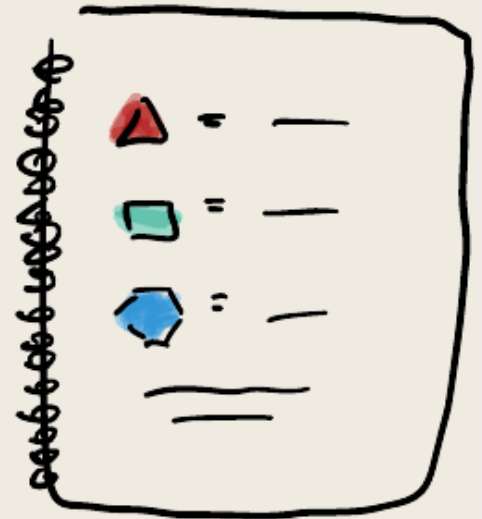
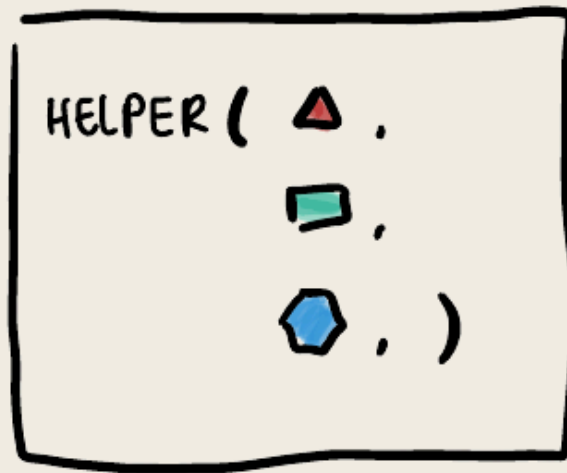
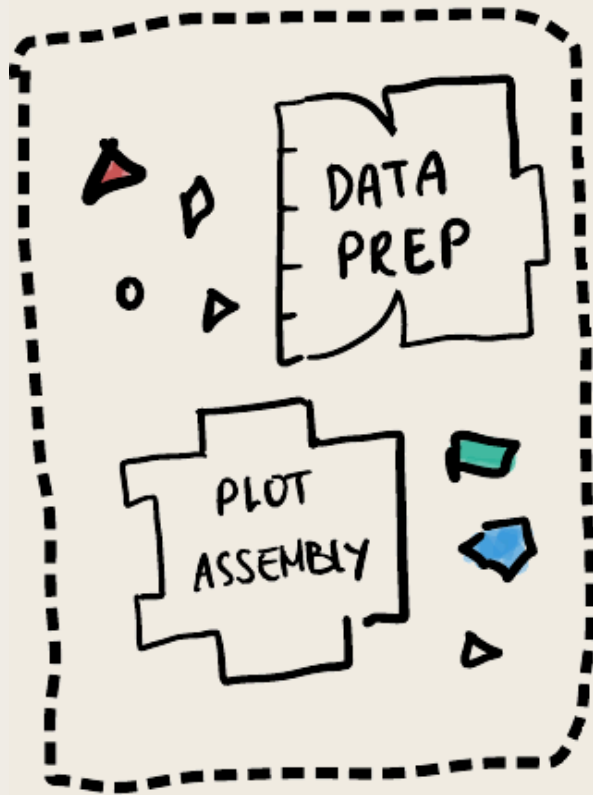
Time is complicated, so is torturing ggplot2 to care about time.

Stay away if you value your sanity.

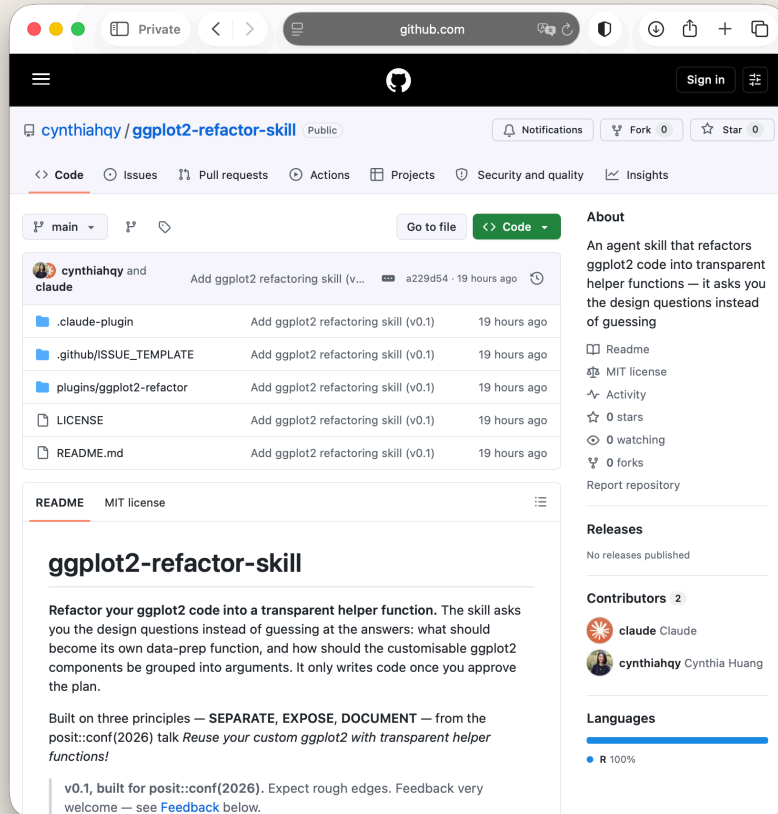


Keep calm, and...

SEPARATE, EXPOSE & DOCUMENT



Thanks for listening!



 Slides, links, skill & more:

github.com/cynthiahqy/positconf2026



Tell me about
your ggplot2 reuse
experiences!

